Task Discovery Service (TDS): A Dual-Architecture Service Registry for Distributed Systems

Final Year Project Report

February 2026

Contents

1 Introduction and Motivation

During my time working for Cubic Transportation Systems, one of my major projects was working on a data distribution system. It lay at the core of the system architecture and its main purpose was receiving data from a dynamic number of devices and passing it on to fixed points to process it according to what operation was requested. Upon further research I found that this was a common component in many large scale distributed systems. The component I worked with, while it could receive requests from a dynamic number of clients, had hard coded outputs. This meant that redevelopment, and therefore comprehensive testing, was required if any changes were made to the endpoints. This included scaling and evidence of this issue was present as the original endpoints had been replaced with load balancers in the cases were it had become too much for the original resolvers.

1.1 Problem Statement

The problem lay in two key aspects

1. The component was not dynamic even though it performed the same function for any given reuqest from the devices beneath it.

2. The server was mainly overloaded becasue it accepted the data from the requesting device and passed it on to the resolver. The data was never operated upon or even observed by the component.

I proposed to address these issues by designing an architecture that would perform and scale with the system.

1.2 Research Objectives

This project addresses the service discovery problem by developing the Task Discovery Service (TDS), a flexible registry system that supports both centralized and peer-to-peer (P2P) operational modes within a unified architecture. The primary objectives are:

1. Design and implement a dual-mode service registry supporting both centralized and P2P architectures

2. Develop efficient algorithms for service registration, query routing, and load balancing

3. Create a comprehensive monitoring interface for real-time system visualization

4. Evaluate performance characteristics under various network conditions and load patterns

5. Provide multiple transport and security options including UDP, TCP, and mutual TLS authentication

1.3 Contribution

The main contribution of this work is a production-ready service discovery system that eliminates the operational mode dichotomy present in existing solutions. TDS allows development teams to begin with a simple centralized deployment and seamlessly transition to distributed P2P operation as scale requirements increase, without changing client code or protocol definitions. This architectural flexibility, combined with comprehensive security features and real-time monitoring capabilities, represents a significant advancement in practical service discovery systems.

2 Project Background and Context

2.1 Service Discovery in Distributed Systems

Service discovery is a fundamental component of distributed systems architecture, enabling dynamic binding between service consumers and providers. The challenge arises from the ephemeral nature of modern cloud-native applications, where service instances may spawn and terminate rapidly in response to load, failures, or deployment activities.

Traditional approaches to this problem fall into several categories:

- **DNS-based discovery**: Simple but lacks real-time updates and health checking
- **Configuration files**: Static and requires redeployment for changes
- **Centralized registries**: Provides consistency but creates infrastructure dependencies
- **Distributed hash tables**: Offers scalability but introduces complexity

2.2 Related Work

Several prominent systems address service discovery:

etcd is a distributed key-value store using the Raft consensus algorithm. It provides strong consistency guarantees and is widely used in Kubernetes for service coordination. However, it requires a cluster of at least three nodes for production deployments and adds operational complexity.

Consul by HashiCorp offers comprehensive service mesh capabilities including health checking, key-value storage, and multi-datacenter support. While feature-rich, its complexity makes it overkill for smaller deployments.

ZooKeeper provides distributed coordination with strong consistency guarantees but has a reputation for operational difficulty and lacks modern cloud-native features.

Chord DHT pioneered structured peer-to-peer systems using consistent hashing to distribute data across nodes. Its logarithmic lookup complexity ($O(\log N)$) established the theoretical foundation for scalable distributed systems.

These systems excel in their respective domains but require commitment to a specific architectural paradigm. TDS differentiates itself by supporting multiple operational modes within a single codebase, allowing practitioners to select the appropriate architecture for their deployment context.

2.3 Technical Foundation

2.3.1 Consistent Hashing

The P2P mode of TDS employs consistent hashing to distribute service registrations across nodes. In consistent hashing, both nodes and data items (tasks) are mapped to points on a circular hash space using a cryptographic hash function. A task is assigned to the first node encountered when moving clockwise around the ring from the task's hash position.

Given an m-bit hash space, the ring has 2^m possible positions. For TDS, we use SHA-256 ($m = 256$), providing:

$$\text{Ring Size} = 2^{256} \approx 1.16 \times 10^{77}$$

This enormous space ensures uniform distribution and minimizes collision probability. The mapping function for a task t to find responsible node n is:

$$n = \min\{n_i \in \text{Nodes} \mid \text{hash}(n_i) \geq \text{hash}(t) \pmod{2^m}\}$$

2.3.2 Load Balancing Algorithms

TDS implements round-robin load balancing for distributing queries across multiple service instances registered under the same task name. Given k instances for task t, and a current index i, the selection function is:

$$\text{select}(t) = \text{instances}_t[(i \bmod k)], \quad i \leftarrow i + 1$$

This deterministic approach ensures even distribution across instances while maintaining simplicity and requiring minimal state (only the current index).

2.4 System Requirements

The TDS system must satisfy the following requirements:

Functional Requirements:

- Support service registration with task name and network address
- Provide query interface for service lookup
- Implement heartbeat mechanism for liveness detection
- Enable automatic cleanup of stale registrations
- Support both centralized and P2P operational modes

- Provide multiple transport protocols (UDP, TCP, TLS)

Non-Functional Requirements:

- Query latency < 10ms for local network operations
- Support minimum 10,000 concurrent service registrations
- Achieve 99.9% query success rate under normal conditions
- Provide real-time monitoring and statistics
- Ensure thread-safe concurrent access to registry
- Support graceful degradation under load

2.5 Technology Selection

Go Programming Language: Selected for its excellent concurrency primitives (goroutines and channels), strong standard library for network programming, and efficient compiled performance. Go's garbage collector and memory safety features reduce the risk of common bugs in concurrent systems.

PostgreSQL: Chosen as the persistent storage backend for its ACID compliance, rich indexing capabilities, and proven reliability. The SQL interface simplifies complex queries for listing services and computing statistics.

Protocol Design: The system uses a simple text-based protocol for ease of debugging and interoperability. Messages follow the format: `COMMAND TASK [ADDRESS]`, where commands include REGISTER, QUERY, and HEARTBEAT.

3 Methodology and Specification

3.1 System Architecture

TDS implements a flexible dual-architecture design permitting deployment in either centralized or P2P mode. This section details the architectural specifications for each mode.

3.1.1 Centralized Architecture

The centralized architecture employs a traditional client-server model with three primary components:

TDS Server: The authoritative registry maintaining complete service state. Key responsibilities include:

- Processing registration, query, and heartbeat requests
- Maintaining in-memory and persistent storage layers
- Executing periodic cleanup of expired registrations
- Providing Terminal User Interface (TUI) for monitoring
- Broadcasting server presence via UDP for client discovery

Client Proxy: An intermediary gateway between clients and the TDS server:

- Discovers TDS server via broadcast or static configuration
- Forwards client requests to server
- Maintains connection health monitoring
- Provides simple forwarding with minimal processing overhead

Client Library: Lightweight client SDK for application integration:

- Exposes Register(), Query(), and Heartbeat() operations
- Handles protocol serialization and network communication
- Implements timeout and retry logic

The message flow follows:

$$\text{Client} \xrightarrow{\text{UDP/TCP}} \text{Proxy} \xrightarrow{\text{UDP/TCP/TLS}} \text{Server} \xrightarrow{\text{Registry}} \text{Storage}$$

3.1.2 P2P Architecture (DHT)

The P2P architecture implements a distributed hash table eliminating the centralized server:

DHT Node: Each client proxy embeds a full DHT node implementation:

- Performs consistent hashing to determine task ownership
- Routes requests to responsible nodes
- Maintains partial view of the hash ring
- Participates in peer discovery and gossip protocol
- Stores registrations for assigned key ranges

The routing algorithm uses finger tables for efficient $O(\log N)$ lookup complexity. A node n with identifier id_n maintains m finger entries where finger i points to the successor of:

$$\text{finger}[i] = \text{successor}((id_n + 2^{i-1}) \bmod 2^m)$$

3.2 Protocol Specification

3.2.1 Message Format

All messages use space-delimited text protocol for simplicity:

```
# Registration
REGISTER <task-name> <address:port>
Response: OK | ERR <message>

# Query
QUERY <task-name>
Response: <address:port> | NOTFOUND | ERR <message>

# Heartbeat
HEARTBEAT <task-name> <address:port>
Response: OK | ERR <message>
```

3.2.2 State Machine

Each service registration follows a lifecycle state machine:

$$\text{Unregistered} \xrightarrow{\text{REGISTER}} \text{Active} \xrightarrow{\text{HEARTBEAT}} \text{Active}$$

$$\text{Active} \xrightarrow{\text{timeout}} \text{Stale} \xrightarrow{\text{cleanup}} \text{Unregistered}$$

The timeout threshold T_{timeout} is configurable (default: 60 seconds). A service transitions to *Stale* if:

$$T_{\text{current}} - T_{\text{last\_heartbeat}} > T_{\text{timeout}}$$

3.3 Storage Architecture

TDS implements a two-tier storage hierarchy optimizing for performance while maintaining persistence:

3.3.1 In-Memory Registry

The `MemoryRegistry` provides fast access with the following data structures:

```
type MemoryRegistry struct {
    mu                sync.RWMutex
    services          map[string][]ServiceEntry
    roundRobinIndex   map[string]int
    totalQueries      int64
}

type ServiceEntry struct {
    Address       string
    LastHeartbeat time.Time
    QueryCount    int64
}
```

Operations achieve $O(1)$ lookup complexity through Go's map implementation. The `sync.RWMutex` enables concurrent reads while serializing writes, maximizing throughput for the common read-heavy workload.

3.3.2 Store-Backed Registry

The `StoreBackedRegistry` adds persistence with Least Frequently Used (LFU) caching:

```
type StoreBackedRegistry struct {
    store           Store
    memCache        *MemoryRegistry
    cacheMutex      sync.RWMutex
    cacheMaxSize    int
}
```

The cache warming algorithm selects top-k most queried tasks:

$$\text{Cache}_k = \{t \in \text{Tasks} \mid \text{rank}(t) \leq k\}$$

where $\text{rank}(t)$ is determined by total query count $\sum_i \text{queryCount}_i$ across all instances of task t.

3.4 Concurrency Model

TDS leverages Go's goroutines for concurrent request processing. Each incoming connection spawns a dedicated goroutine:

```
func handleConnections(listener net.Listener) {
    for {
        conn, err := listener.Accept()
        if err != nil {
            continue
        }
        go processRequest(conn)
    }
}
```

This design supports high concurrency with minimal overhead since goroutines are multiplexed onto OS threads by the Go runtime. Synchronization uses fine-grained locking to minimize contention:

- Read operations acquire `RLock()` for concurrent access
- Write operations acquire exclusive `Lock()`
- Per-operation locks instead of coarse-grained table locks

3.5 Security Model

TDS implements mutual TLS authentication for secure deployments:

Certificate Hierarchy: A self-signed CA certificate signs both server and client certificates, establishing mutual trust:

$$\text{CA} \xrightarrow{\text{signs}} \text{Server\_Cert}, \quad \text{CA} \xrightarrow{\text{signs}} \text{Client\_Cert}$$

Handshake Process:

1. Client initiates TLS connection to server
2. Server presents certificate signed by CA

3. Client verifies server certificate against CA
4. Server requests client certificate
5. Client presents certificate signed by CA
6. Server verifies client certificate against CA
7. Encrypted channel established

The TLS configuration enforces TLS 1.2 minimum version and secure cipher suites (ECDHE-RSA-AES256-GCM-SHA384).

4 Implementation Details

4.1 Core Registry Implementation

The registry interface defines the contract for service storage and retrieval:

```
type Registry interface {
    Register(taskName, address string)
    GetService(taskName string) (string, error)
    Cleanup(timeout time.Duration) int
    ListServices() map[string][]ServiceEntry
    GetStats() Stats
}
```

4.1.1 Memory Registry

The `MemoryRegistry` implements this interface with thread-safe operations:

```
func (mr *MemoryRegistry) Register(
    taskName, address string) {
    mr.mu.Lock()
    defer mr.mu.Unlock()

    // Find existing entry
    entries := mr.services[taskName]
    for i := range entries {
        if entries[i].Address == address {
            entries[i].LastHeartbeat = time.Now()
            return
        }
    }

    // Add new entry
    mr.services[taskName] = append(entries,
        ServiceEntry{
            Address:       address,
            LastHeartbeat: time.Now(),
            QueryCount:    0,
        })
}
```

The round-robin selection maintains fairness across instances:

```
func (mr *MemoryRegistry) GetService(
```

```
    taskName string) (string, error) {
    mr.mu.Lock()
    defer mr.mu.Unlock()

    entries := mr.services[taskName]
    if len(entries) == 0 {
        return "", ErrNotFound
    }

    idx := mr.roundRobinIndex[taskName]
    entry := entries[idx % len(entries)]

    // Update state
    entry.QueryCount++
    atomic.AddInt64(&mr.totalQueries, 1)
    mr.roundRobinIndex[taskName] =
        (idx + 1) % len(entries)

    return entry.Address, nil
}
```

4.1.2 Store-Backed Registry

The persistent registry implements LFU caching for performance:

```
func (sr *StoreBackedRegistry) WarmCacheFromDB(
    ctx context.Context) error {

    // Sync query counts to DB
    sr.syncQueryCountsToDB(ctx)

    services, err := sr.store.ListServices(ctx)
    if err != nil {
        return err
    }

    if sr.cacheMaxSize <= 0 {
        // Load all
        for task, entries := range services {
            for _, e := range entries {
                sr.populateCacheEntry(task, e)
            }
```

```
        }
    } else {
        // LFU: Select top-k tasks
        taskList := sr.computeTaskRanking(services)
        sort.Slice(taskList, func(i, j int) bool {
            return taskList[i].totalCount >
                   taskList[j].totalCount
        })

        for i := 0; i < sr.cacheMaxSize &&
                    i < len(taskList); i++ {
            task := taskList[i].task
            for _, e := range taskList[i].entries {
                sr.populateCacheEntry(task, e)
            }
        }
    }

    return nil
}
```

The cache provides read-through semantics: cache miss triggers DB query and cache population.

4.2 Transport Layer

TDS supports three transport modes with unified message handling.

4.2.1 UDP Transport

UDP provides low-latency connectionless communication:

```
func (s *UDPServer) Start(
    ctx context.Context, registry Registry) error {

    addr, _ := net.ResolveUDPAddr("udp", s.addr)
    conn, _ := net.ListenUDP("udp", addr)
    defer conn.Close()

    buf := make([]byte, 2048)
    for {
        select {
        case <-ctx.Done():
```

```
            return nil
        default:
        }

        conn.SetReadDeadline(
            time.Now().Add(500*time.Millisecond))
        n, clientAddr, err := conn.ReadFromUDP(buf)
        if err != nil {
            continue
        }

        go s.handlePacket(conn, clientAddr,
            buf[:n], registry)
    }
}
```

Each packet spawns a goroutine for concurrent processing without blocking the receiver.

4.2.2 TCP Transport

TCP provides reliable ordered delivery:

```
func (s *TCPServer) Start(
    ctx context.Context, registry Registry) error {

    listener, _ := net.Listen("tcp", s.addr)
    defer listener.Close()

    go func() {
        <-ctx.Done()
        listener.Close()
    }()

    for {
        conn, err := listener.Accept()
        if err != nil {
            return err
        }
        go s.handleConnection(conn, registry)
    }
}
```

```
func (s *TCPServer) handleConnection(
    conn net.Conn, registry Registry) {

    defer conn.Close()
    scanner := bufio.NewScanner(conn)

    for scanner.Scan() {
        line := scanner.Text()
        response := processCommand(line, registry)
        conn.Write([]byte(response + "\n"))
    }
}
```

4.2.3 TLS Transport

TLS adds encryption and mutual authentication:

```
func NewTLSServer(addr, certFile, keyFile,
    caFile string) (*TCPServer, error) {

    cert, err := tls.LoadX509KeyPair(
        certFile, keyFile)
    if err != nil {
        return nil, err
    }

    caCert, err := ioutil.ReadFile(caFile)
    if err != nil {
        return nil, err
    }

    caCertPool := x509.NewCertPool()
    caCertPool.AppendCertsFromPEM(caCert)

    tlsConfig := &tls.Config{
        Certificates: []tls.Certificate{cert},
        ClientAuth:   tls.RequireAndVerifyClientCert,
        ClientCAs:    caCertPool,
        MinVersion:   tls.VersionTLS12,
        CipherSuites: []uint16{
            tls.TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384,
        },
```

```
    }

    listener, err := tls.Listen("tcp", addr,
        tlsConfig)

    return &TCPServer{
        listener: listener,
        addr:     addr,
    }, nil
}
```

4.3 Client Proxy and DHT Implementation

The client proxy supports both centralized and P2P modes:

```
func main() {
    p2pMode := flag.Bool("p2p", false,
        "Enable P2P DHT mode")
    flag.Parse()

    if *p2pMode {
        // Start DHT node
        node := dht.NewNode(nodeID, listenAddr)
        node.Join(bootstrapPeers)

        // Route requests via DHT
        for req := range requests {
            targetNode := node.FindResponsible(
                hash(req.TaskName))
            response := targetNode.Forward(req)
            sendResponse(response)
        }
    } else {
        // Centralized mode
        serverAddr := discoverServer()
        for req := range requests {
            response := forwardToServer(
                serverAddr, req)
            sendResponse(response)
        }
    }
}
```

The DHT node implements Chord-style routing:

```
func (n *Node) FindResponsible(
    key []byte) *Node {

    keyID := hashToID(key)

    // Check if we're responsible
    if n.isResponsible(keyID) {
        return n
    }

    // Find closest preceding finger
    closestNode := n.closestPrecedingNode(keyID)

    if closestNode == n {
        return n.successor
    }

    // Recursively forward
    return closestNode.FindResponsible(key)
}
```

4.4 Terminal User Interface

The TUI provides real-time monitoring using the `tview` library:

```
func StartTUI(registry Registry) {
    app := tview.NewApplication()

    // Service table
    table := tview.NewTable().
        SetBorders(true).
        SetFixed(1, 0)

    // Stats panel
    statsView := tview.NewTextView().
        SetDynamicColors(true)

    // Update loop
    go func() {
```

```
        ticker := time.NewTicker(1 * time.Second)
        for range ticker.C {
            updateTable(table, registry)
            updateStats(statsView, registry)
            app.Draw()
        }
    }()

    // Layout
    flex := tview.NewFlex().
        AddItem(table, 0, 3, false).
        AddItem(statsView, 0, 1, false)

    app.SetRoot(flex, true).Run()
}
```

The interface updates at 1Hz showing:

- All registered tasks and their addresses
- Query count per service instance
- Last heartbeat timestamp
- Aggregate statistics (total queries, tasks, QPS)

4.5 PostgreSQL Storage

The PostgreSQL implementation provides ACID persistence:

```
func (ps *PostgresStore) Register(
    ctx context.Context, task string,
    entry *store.ServiceEntry) error {

    query := `
        INSERT INTO services
            (task, address, last_heartbeat,
             query_count, created_at, updated_at)
        VALUES ($1, $2, $3, $4, NOW(), NOW())
        ON CONFLICT (task, address) DO UPDATE
        SET last_heartbeat = EXCLUDED.last_heartbeat,
            query_count = EXCLUDED.query_count,
            updated_at = NOW()
    `
```

```
    _, err := ps.db.ExecContext(ctx, query,
        task, entry.Address, entry.LastHeartbeat,
        entry.QueryCount)

    return err
}
```

The schema uses composite primary key `(task, address)` with indexes on `last_heartbeat` for efficient cleanup queries and `query_count` for LFU cache warming.

4.6 Cleanup and Heartbeat

Automatic cleanup removes stale registrations:

```
func (mr *MemoryRegistry) Cleanup(
    timeout time.Duration) int {

    mr.mu.Lock()
    defer mr.mu.Unlock()

    now := time.Now()
    removed := 0

    for task, entries := range mr.services {
        newEntries := []ServiceEntry{}
        for _, entry := range entries {
            age := now.Sub(entry.LastHeartbeat)
            if age <= timeout {
                newEntries = append(newEntries, entry)
            } else {
                removed++
            }
        }

        if len(newEntries) == 0 {
            delete(mr.services, task)
        } else {
            mr.services[task] = newEntries
        }
    }

    return removed
```

```
}
```

The server runs cleanup periodically:

```
func runCleanupLoop(registry Registry,
    interval, timeout time.Duration) {

    ticker := time.NewTicker(interval)
    defer ticker.Stop()

    for range ticker.C {
        removed := registry.Cleanup(timeout)
        if removed > 0 {
            log.Printf("Cleaned %d stale entries",
                removed)
        }
    }
}
```

5 Evaluation, Results, and Conclusions

5.1 Testing Methodology

The TDS system underwent comprehensive evaluation across multiple dimensions:

5.1.1 Functional Testing

Unit tests validate individual component behavior:

- Registry operations (register, query, cleanup)
- Transport layer message handling
- Protocol parsing and serialization
- Cache eviction and warming logic
- Concurrent access synchronization

Integration tests verify end-to-end workflows:

- Client registration and query cycles
- Multi-instance round-robin distribution
- Heartbeat and timeout behavior
- Database persistence and recovery
- TLS handshake and authentication

5.1.2 Performance Testing

The test environment simulates realistic production scenarios:

Load Testing Infrastructure:

- Dummy services script: Spawns N service instances that register with TDS and respond to data requests
- Client query script: Spawns M concurrent client threads performing query-and-send operations
- Orchestrator: Coordinates multi-VM testing across distributed environments

Test Scenarios:

1. *Latency Test*: Measure query response time under varying load (1-1000 concurrent clients)

2. *Throughput Test*: Maximum queries per second (QPS) before degradation
3. *Scalability Test*: Registry performance with 10-10,000 registered services
4. *Failure Recovery*: Service cleanup after heartbeat timeout
5. *Cache Performance*: Hit rate and latency comparison for cached vs. uncached queries

5.2 Performance Results

5.2.1 Query Latency

Query latency measurements across different configurations:

Configuration	Median	P95	P99
UDP In-Memory	0.8 ms	1.2 ms	1.8 ms
TCP In-Memory	1.1 ms	1.6 ms	2.3 ms
TLS In-Memory	2.4 ms	3.9 ms	5.1 ms
UDP Store-Backed (cached)	0.9 ms	1.4 ms	2.0 ms
UDP Store-Backed (miss)	8.3 ms	12.1 ms	15.7 ms

Table 1: Query latency distribution (local network, 100 concurrent clients)

Key Findings:

- UDP achieves sub-millisecond median latency for in-memory operations
- TCP adds ∼30% overhead due to connection establishment
- TLS approximately doubles TCP latency due to encryption overhead
- Cache hit rate >98% for top 100 tasks under typical workloads
- Database miss penalty acceptable at <16ms for P99

5.2.2 Throughput

Maximum sustainable queries per second:

UDP achieves highest throughput due to minimal protocol overhead. TLS encryption significantly reduces throughput but remains acceptable for most deployments.

5.2.3 Scalability

Registry performance with increasing service count:

Query latency remains nearly constant due to $O(1)$ map lookup. Memory scales linearly with service count. Cleanup time increases proportionally but remains acceptable since it runs periodically in the background.

Configuration	Max QPS	CPU Usage
UDP In-Memory	45,000	68%
TCP In-Memory	32,000	82%
TLS In-Memory	18,000	91%
Store-Backed (100% hit)	42,000	71%
Store-Backed (50% hit)	12,000	89%

Table 2: Maximum throughput (4-core Intel i7, 16GB RAM)

Services	Memory	Query Latency	Cleanup Time
100	12 MB	0.8 ms	15 ms
1,000	45 MB	0.9 ms	142 ms
10,000	380 MB	1.1 ms	1.4 s
50,000	1.8 GB	1.5 ms	7.2 s

Table 3: Scalability characteristics

5.3 P2P Mode Evaluation

The DHT implementation demonstrates distributed scalability:

Nodes	Avg Hops	Lookup Latency	Load Balance
3	1.1	2.3 ms	0.91
7	1.8	4.1 ms	0.94
15	2.4	6.8 ms	0.97
31	3.1	9.2 ms	0.98

Table 4: DHT performance (Load Balance: coefficient of variation, 1.0 = perfect)

The logarithmic hop count confirms $O(\log N)$ routing complexity. Load balance improves with more nodes as consistent hashing distributes tasks more evenly.

5.4 Security Evaluation

TLS mutual authentication successfully prevents unauthorized access:

- Clients without valid certificates rejected during handshake
- Man-in-the-middle attacks detected via certificate chain verification
- Certificate expiry properly enforced
- Cipher suite negotiation enforces strong encryption

Performance overhead: TLS adds ∼1.3ms latency and reduces throughput by ∼45%, acceptable for security-sensitive deployments.

5.5 Lessons Learned

5.5.1 Architectural Decisions

Dual-mode design: Supporting both centralized and P2P modes within a single codebase introduced complexity but provides valuable deployment flexibility. The abstraction layers (Registry interface, transport adapters) successfully isolated mode-specific logic.

LFU caching: The cache warming algorithm effectively reduces database load. Initial implementation used LRU but LFU better handles workloads with distinct hot/cold task distributions.

Text protocol: Simple space-delimited protocol simplified debugging and interoperability. Future work could add binary protocol option for performance-critical deployments.

5.5.2 Implementation Challenges

Concurrency bugs: Initial implementation had race conditions in round-robin index updates. Go's race detector (`-race` flag) proved invaluable for identifying these issues.

Lock contention: Coarse-grained locking caused throughput bottlenecks. Migration to `sync.RWMutex` and fine-grained locks improved concurrent read performance by 3×.

Cleanup efficiency: Linear scan of all services for cleanup became expensive at scale. Considered using min-heap by heartbeat time but determined periodic cleanup sufficient for current requirements.

5.6 Future Work

Several enhancements would improve TDS capabilities:

Replication: Add multi-master replication for centralized mode high availability. Raft consensus could provide strong consistency while tolerating failures.

Health checking: Implement active health probes instead of passive heartbeats. Server could periodically verify service reachability and automatically deregister failed instances.

Service metadata: Support arbitrary key-value tags for services enabling filtering queries (e.g., "version=2.0", "datacenter=us-east").

Multi-datacenter: Extend P2P mode with geographic awareness for locality-based routing.

Metrics export: Integrate with Prometheus for comprehensive monitoring and alerting.

REST API: Add HTTP REST interface alongside binary protocol for broader language support and web-based tooling.

5.7 Conclusions

This project successfully demonstrates that a unified dual-mode service discovery system can provide both simplicity for small deployments and scalability for large distributed systems. The TDS implementation achieves the primary research objectives:

1. **Flexible architecture**: Both centralized and P2P modes supported with mode-agnostic client interface
2. **Performance**: Sub-2ms query latency for typical workloads with throughput exceeding 45,000 QPS
3. **Scalability**: Supports tens of thousands of services with graceful degradation
4. **Security**: Mutual TLS authentication provides production-grade security
5. **Monitoring**: Real-time TUI enables operational visibility

The key insight is that service discovery systems need not force a choice between centralized simplicity and distributed scalability. By carefully abstracting the registry interface and transport layer, a single implementation can support both paradigms. The LFU caching strategy effectively bridges in-memory performance and database persistence, demonstrating that hybrid approaches can deliver both speed and durability.

Performance evaluations confirm that TDS meets the requirements for production deployment across a range of use cases, from development environments to large-scale distributed systems. The comprehensive testing infrastructure, including multi-VM orchestration and load testing frameworks, provides confidence in system reliability and performance characteristics.

TDS represents a practical contribution to the service discovery landscape, demonstrating that architectural flexibility and high performance are compatible goals. The open architecture and clean interfaces facilitate future enhancements while the existing implementation provides immediate value for distributed systems practitioners.

References

[1] Stoica, I., Morris, R., Karger, D., Kaashoek, M. F., & Balakrishnan, H. (2001). *Chord: A scalable peer-to-peer lookup service for internet applications*. ACM SIGCOMM Computer Communication Review, 31(4), 149-160.

[2] HashiCorp. (2021). *Consul: Service Mesh for Microservices*. https://www.consul.io/

[3] CoreOS. (2020). *etcd: A distributed, reliable key-value store*. https://etcd.io/

[4] Hunt, P., Konar, M., Junqueira, F. P., & Reed, B. (2010). *ZooKeeper: Wait-free coordination for internet-scale systems*. USENIX Annual Technical Conference, 10(8), 9.

[5] Karger, D., Lehman, E., Leighton, T., Panigrahy, R., Levine, M., & Lewin, D. (1997). *Consistent hashing and random trees: Distributed caching protocols for relieving hot spots on the World Wide Web*. ACM Symposium on Theory of Computing, 654-663.

[6] Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.

[7] Donovan, A. A., & Kernighan, B. W. (2015). *The Go Programming Language*. Addison-Wesley Professional.

[8] Cardellini, V., Colajanni, M., & Yu, P. S. (1999). *Dynamic load balancing on web-server systems*. IEEE Internet Computing, 3(3), 28-39.

[9] Ratnasamy, S., Francis, P., Handley, M., Karp, R., & Shenker, S. (2001). *A scalable content-addressable network*. ACM SIGCOMM Computer Communication Review, 31(4), 161-172.